EE650 · BASICS OF MODERN CONTROL SYSTEMS · PROGRESS REVIEW

Fault-tolerant control of a quadrotor after complete rotor loss

Physics-informed residual Koopman model predictive control

Pranshu Kumar · Mudita Jain · Prashant Gupta · Aarush Singh Kushwaha

October 2026

THE PROBLEM

What happens when a rotor fails

A quadrotor uses its four thrusts to control four things: height, roll, pitch and yaw. With one rotor gone, the twisting (reaction) torques of the propellers no longer cancel, so the drone cannot avoid spinning.

Healthy flight
Four rotors, normal controller

→

Rotor fails
One thrust lost; yaw torque no longer balanced

→

Drone spins up
Spin rate grows towards ≈ 19 rad/s

→

Failure controller
Holds the spin axis and the height

→

Relaxed hover
Steady spin, axis tilted 16°

Giving up yaw keeps the drone flying.

It hovers like a spinning top and moves by tilting its spin axis. Mueller & D'Andrea (2014) derived this "relaxed hover", showed that it is controllable and flew it.

PROBLEM STATEMENT

The physics model is good, but not perfect

WHAT THE CONTROLLER RELIES ON

The spinning-drone model of Mueller & D'Andrea: correct equilibrium, linear about the spin, spin-up time matched in flight.

WHAT THE MODEL MISSES

In flight: less thrust while spinning fast, and a spin about 20–27% slower than predicted. Also inertia errors and motor speed. A controller built on it settles with a steady error.

RESEARCH QUESTION

Can a data-driven model of only the mismatch, built on top of the physics model, predict the failed drone more accurately and give a smaller steady error in MPC than the physics model alone or a Koopman model of the full dynamics?

Koopman / EDMD: describe the state by many nonlinear features (here all pairwise products) and fit a linear model on them by least squares. We use the same tools to learn the one-step error of the physics model; it is evaluated outside the MPC optimization, so the MPC stays a convex quadratic program (QP). **Gap:** residual Koopman models exist for healthy drones and other robots (Todde et al. 2026; Bruder et al. 2025); we did not find them for the spinning regime after complete rotor loss.

Simulator and vehicle

- 6-DOF rigid body, quaternion attitude, rotor gyroscopic terms
- Per-rotor thrust 0.2–3.8 N; a motor can also be stopped
- First-order motor lag, 15 ms
- Reduced attitude n: the "up" direction seen from the body (all that is controlled after the failure)
- RK4 integration; controller sample period 1–5 ms, command held between samples
- "Truth" plants for learning data: same simulator plus effects the design model lacks (thrust loss at high spin, inertia error, slower motors)

Parameter	Value
Mass	0.50 kg
Arm length	0.17 m
Inertia Ixx, Izz	3.2, 5.5×10^{-3} kg m^2
Thrust coefficient κf ($f = \kappa f\, \Omega^2$)	6.41×10^{-6} N s^2/rad^2
Yaw torque per newton $\kappa\tau$	0.0169 m
Yaw air drag γ	2.75×10^{-3} N m s/rad

All values from Mueller & D'Andrea, ICRA 2014, Sec. V-A

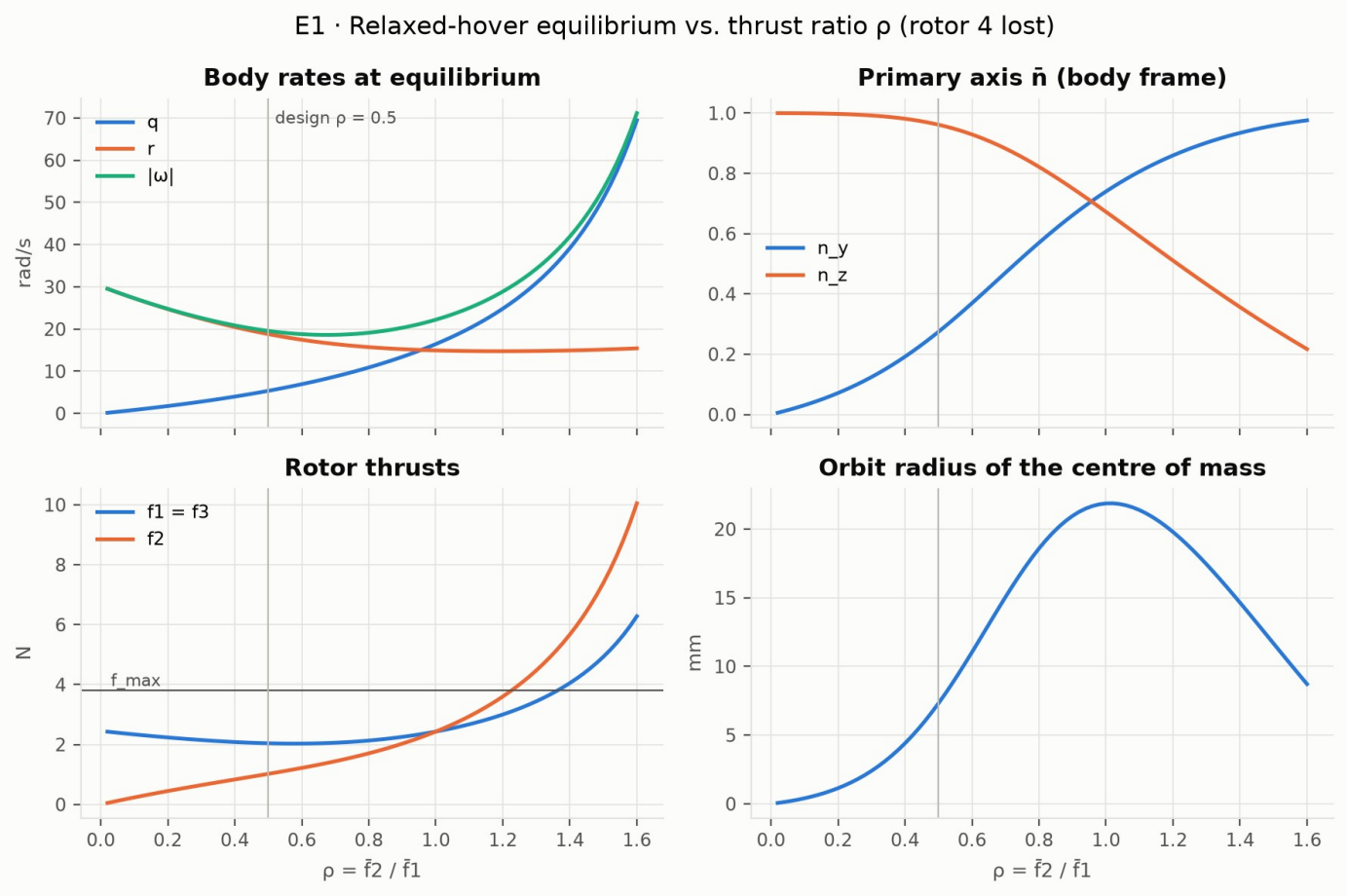
FOUNDATION

The model matches three published papers

Source	What we compared	Result
Mueller & D'Andrea 2014	Two- and three-rotor-loss equilibria	Every printed digit (≤ 0.8%)
Mueller & D'Andrea 2014	One-rotor-loss equilibrium	Thrusts, yaw rate, n_z ≤ 0.5%; pitch rate −6% *
Mueller & D'Andrea 2016	Their more detailed model, 5 hover cases (separate code)	Body rates within 4%
Sun et al. 2021	Second drone (Parrot Bebop 2), two rotors lost	Equal to the paper's formula
Flight data	Spin-up to 10 rad/s at the flight thrust (2.95 N)	**0.65 s vs 0.62 s in flight (4%)**

* The paper's own published values do not satisfy its own roll equation; its other two cases match exactly. **47 automated tests** (46 pass; 1 expected failure keeps that inconsistency visible) also check conservation laws, closed-form solutions and these published numbers.

The spinning equilibrium



Rotor 4 lost. $f_1 = f_3$: the two side rotors; f_2: the rotor opposite the failed one. Design ratio $\rho = f_2/f_1 = 0.5$:

f = (2.04, 1.02, 2.04) N
spin 19.6 rad/s, axis tilted 16°

Thrusts, yaw rate and n_z match the paper within 0.5%; pitch rate and tilt are about 5% lower (see validation).

The figure shows how the equilibrium changes with ρ; it reproduces the paper's Fig. 3 (minimum spin 18.6 vs 19.0 rad/s).

Linear model: controllable, but what can we observe?

7 states (p, q, r, n_x, n_y, z, v_z), inputs f_1, f_2, f_3. Linear and time-invariant in body axes.

Open-loop modes: ±10.1j, ±19.6j, −0.51 and an altitude double integrator.

Controllable: the three rotors can steer all 7 states (controllability rank 7 of 7; every mode can be influenced, PBH test).

Sensors	Observable (of 7)	Cannot see
Gyro	3 / 7	tilt, altitude
Gyro + tilt estimate	5 / 7	altitude
Gyro + barometer	7 / 7	none (weak)
Gyro + tilt + barometer	7 / 7	none

The equilibrium is tilted, so vertical acceleration depends on tilt. A barometer then reveals the tilt in principle, but with realistic sensor noise the weakest direction is about 2,700× less visible than with a tilt sensor (smallest observability-Gramian eigenvalue).

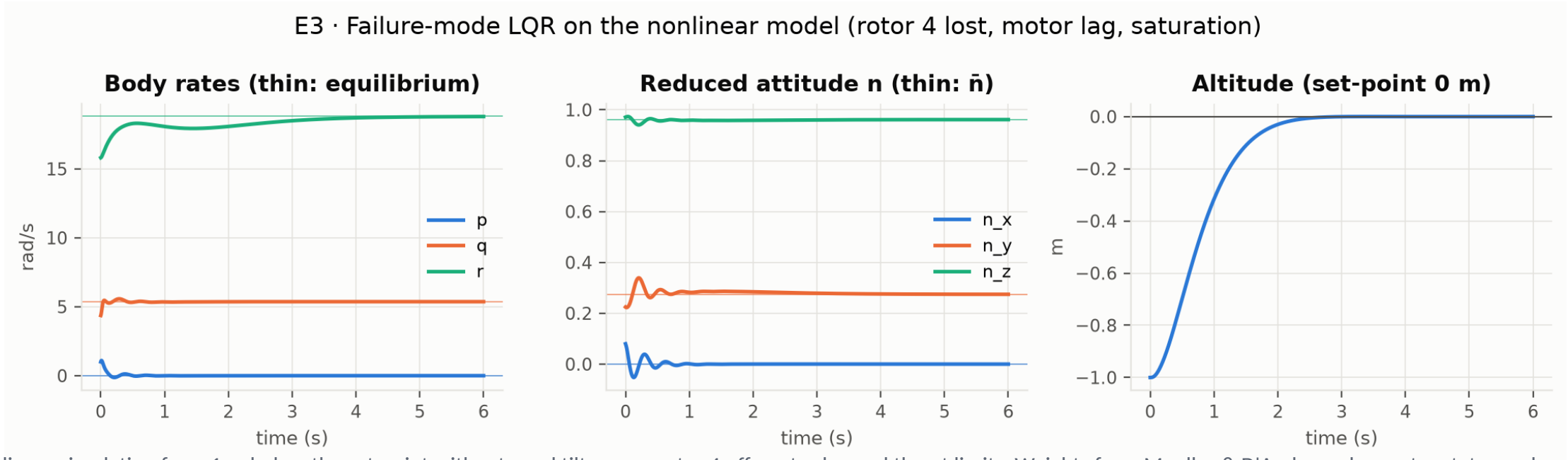
FOUNDATION

LQR baseline at the spinning equilibrium

18 ms
input-delay margin (healthy LQR: 40 ms)

ζ ≈ 0.19
damping of the least-damped mode

1.8–2.5 s
to settle altitude, tilt and yaw rate



Nonlinear simulation from 1 m below the set-point with rate and tilt errors, rotor 4 off, motor lag and thrust limits. Weights from Mueller & D'Andrea, plus motor states and altitude. The delay margin is less than half the healthy value: the spinning drone is much less forgiving.

THE APPROACH

Residual Koopman models of the failed vehicle

Learn only what the physics model gets wrong, then use it inside a convex MPC.

SET-UP

Three predictors, same data

Analytical

Mueller & D'Andrea linear model about the spinning equilibrium. No data.

$x^{+} = A_d\, x + B_d\, u$

Full-state EDMDc

Learns everything: 55 features of the state (constant, states, all pairwise products), linear model on them.

$\zeta^{+} = A\, \zeta + B\, u$

Residual (proposed)

Physics model plus a learned one-step correction, clipped at d_max (99.5th percentile of the training errors).

$x^{+} = A_d\, x + B_d\, u + \mathrm{clip}(W^{T}\psi(x, u))$

Data: 40 episodes × 4 s at 5 ms, from a truth plant with effects the model lacks (thrust loss at high spin, inertia error, slower motors). **Test:** 10 new episodes from the same plant (A) and 10 from a different one (B).

From 100 ms to 1 s, the residual model predicts best



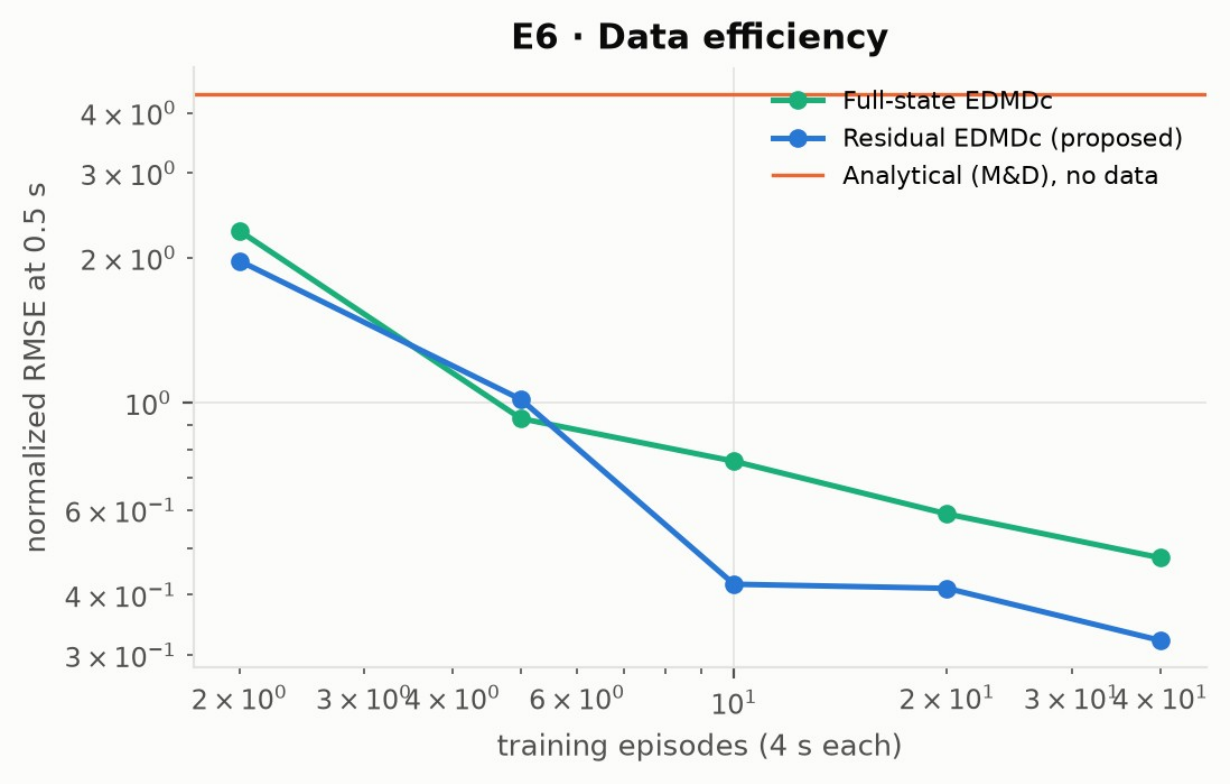
Error at a 1 s horizon on new data from plant A (normalized RMSE: error ÷ each state's typical spread; 1 = as large as normal variation):

Predictor	Error
Residual EDMDc	**0.64**
Full-state EDMDc	0.91
"No change" baseline	2.38
Analytical model	7.09

Below 100 ms the full-state model is better (2.5× at 20 ms); it has 4 of 55 eigenvalues just outside the unit circle and falls behind "no change" by 2 s, as we hypothesised. Beyond about 2 s "no change" beats every model. The physics model drifts in open loop (undamped modes, unmodelled thrust loss) but works in closed loop.

LIMITS

Little data needed, no generalization yet



Data efficiency

With 10 episodes the residual model (0.42) beats the full-state model trained on 40 (0.48), at a 0.5 s horizon.

Generalization fails

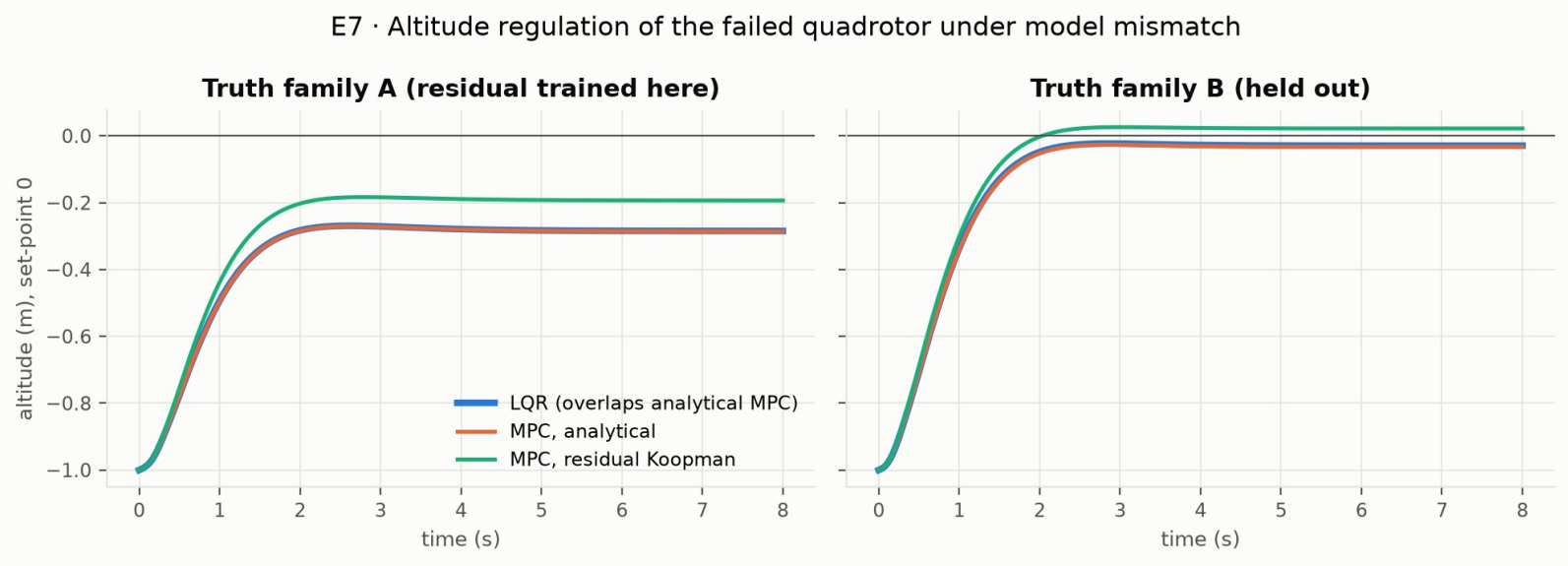
On the unseen plant B, both learned models are worse than the physics model alone. Training on randomized plants helps on B only up to about 100 ms and hurts on A: a model that sees only the current state cannot tell which vehicle it is on.

Next

Update the residual online with recursive least squares (it is linear in its parameters).

CONTROL

Residual MPC removes a third of the bias



−32%

settles 19 cm low instead of 29 cm (plant A)

0.7 ms

QP solver per 5 ms step (4.4 ms including the Python modelling layer)

The residual is evaluated outside the QP and added as a known disturbance, so the problem stays convex (as in Todde et al. 2026). It helps for every mismatch effect of plant A; on held-out plant B the correction can push the wrong way (one variant: 47% larger error). **Missing baseline: integral action (offset-free MPC), which removes a constant bias completely in theory.**

NEXT STEPS

Plan for the remaining weeks

Dates	Work
12–18 Oct	MPC on the full-state Koopman model (the second controller of the proposal); integral-action (offset-free) MPC baseline
19–25 Oct	Online adaptation of the residual by recursive least squares, to fix generalization
26–31 Oct	Air-relative thrust in the truth plant; residual trained on spin-up transients; presentation 31 Oct
1–13 Nov	Final simulations and the journal-style report, due 13 Nov

Main risk: online adaptation may need richer data than one flight provides. Fallback: a residual that also uses a short history of states.

SUMMARY

Where we stand

1 The simulator matches three published papers and the flight spin-up time within 4%.

2 Learning only the error of the physics model predicts best from 100 ms to 1 s; with 10 training episodes it beats a full Koopman model trained on 40.

3 Inside MPC it cuts the altitude bias by a third, but does not yet generalize to an unseen vehicle. Online adaptation is next.

Questions?

BACKUP

Key references

- M. W. Mueller, R. D'Andrea. Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers. ICRA 2014.
- M. W. Mueller, R. D'Andrea. Relaxed hover solutions for multicopters. IJRR 35(8), 2016.
- S. Sun, X. Wang, Q. Chu, C. de Visser. Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors. IEEE T-RO 37(1), 2021.
- M. Korda, I. Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. Automatica 93, 2018.
- M. O. Williams, I. G. Kevrekidis, C. W. Rowley. A data-driven approximation of the Koopman operator: extending dynamic mode decomposition. J. Nonlinear Sci. 25, 2015.
- E. Todde, S. Martini, A. Rizzo, K. P. Valavanis. Residual Koopman-based model predictive control of quadrotors. ICUAS 2026.
- D. Bruder, D. Bombara, R. J. Wood. A Koopman-based residual modeling approach for the control of a soft robot arm. IJRR 44(3), 2025.
- S. S. K. S. Narayanan, U. Vaidya. RK-MPC: residual Koopman MPC for quadruped locomotion in offroad environments. arXiv:2604.04221, 2026.
- B. Stellato et al. OSQP: an operator splitting solver for quadratic programs. Math. Prog. Comp. 12, 2020.

Full bibliography: the project report.